

VanSetu Development Guide

VanSetu — Development Guide

Companion file to `VanSetu_Blueprint_v2.md`. This is the working reference for the build itself — phase checkpoints, conventions, and definitions of “done.” Send this alongside the blueprint to Antigravity (or any dev environment/collaborator).

1. Project Identity

- **Name:** VanSetu
- **Problem statement:** SIH12508 — Ministry of Tribal Affairs — AI-powered FRA Atlas & WebGIS-based DSS
- **Context:** Built for a college SIH internal selection round (100 teams → 10 shortlisted)
- **Full technical spec:** see `VanSetu_Blueprint_v2.md` — this file does not repeat schema/API details, only process and conventions

2. Ground Rules (apply to every phase)

1. **Free tier only.** No paid API keys, no services requiring a card on file. If a free option isn't good enough, flag it — don't silently switch to a paid one.
2. **No silent AI outputs.** Risk scores, conflict detection, evidence checks, OCR results, scheme recommendations — every one of these must show *why*, not just a number or a verdict. If a feature can't explain itself, it's not done yet.
3. **No fake status.** Anything shown as “ACTIVE,” “verified,” or “complete” in the UI must be genuinely implemented and working — this applies especially to the Trust Center / security panel (blueprint §17).
4. **Synthetic data only.** No real claimant names, real GPS coordinates of real people, or real documents — ever, at any phase.
5. **Server-side authority.** The frontend never talks to the database directly, and never trusts its own role checks — every permission check happens again on the backend.
6. **Label prototype logic as prototype logic.** Conflict severity thresholds and risk score weights are our own design choices, not official FRA rules — say so in the UI copy, not just the docs.

3. Phase Checklist & Definition of Done

Work through phases **in order** — each one should be genuinely working before starting the next. “Working” means you can demonstrate it live, not just that the code compiles.

Phase 1 — Foundation

- ☐ Repo structure set up (frontend/backend split, or monorepo — document the choice)
- ☐ `.env.example` present, no real secrets committed
- ☐ PostgreSQL + PostGIS running locally

- ☐ Core schema migrated: users, claims, evidence, conflicts, audit_logs (see blueprint §4)
- ☐ Auth working: register, login, JWT issuance, refresh, logout
- ☐ RBAC enforced server-side for all 4 roles (Citizen, Field Officer, District Admin, State Admin)
- **Done when:** you can register a user of each role, log in, and hit a protected test route that correctly allows/denies based on role.

Phase 2 — GIS

- ☐ Leaflet map renders with district/tehsil/village boundary layers
- ☐ Claim polygons load from PostGIS and render on the map
- ☐ Filters work: state, district, tehsil, village, claim type, status, risk level
- ☐ Marker clustering / geometry simplification in place for 2,000+ claim performance (blueprint §6)
- **Done when:** the map loads real seeded polygon data, filters correctly, and doesn't lag when zoomed out to state level.

Phase 3 — Claim Engine

- ☐ Claim CRUD (create/read/update/delete) with full status workflow (blueprint §5)
- ☐ Evidence upload with server-side MIME verification
- ☐ Claim detail page: number, type, area, status, timeline, actions
- ☐ Audit log writes on every claim mutation
- **Done when:** you can create a claim end-to-end, attach evidence, change its status, and see it reflected in both the map and an audit log entry.

Phase 4 — Conflict Engine

- ☐ PostGIS ST_Intersects / ST_Intersection overlap detection working
- ☐ Overlap % calculated and severity banded (LOW/MEDIUM/HIGH, clearly labeled as prototype thresholds)
- ☐ Conflict review queue + resolution workflow
- ☐ Conflict heatmap toggle on the Atlas
- **Done when:** creating two overlapping claim polygons automatically produces a conflict record with the correct overlap % — verify this with a unit test, not just visually.

Phase 5 — Risk Engine

- ☐ Risk score calculated from the defined factors (blueprint §8), weights clearly labeled as prototype-only
- ☐ Priority queue sorted by risk score
- ☐ “Why This Claim?” explainable breakdown showing the factor-by-factor math
- ☐ Evidence Completeness Checker feeding into the score (blueprint §14)
- **Done when:** a claim with a conflict + missing evidence + long pending time visibly lands in the HIGH/CRITICAL band, and clicking into it shows the correct factor breakdown.

Phase 6 — Offline Mode

- ☐ PWA installable, service worker registered
- ☐ IndexedDB/Dexie caching for assigned claims, encrypted at rest
- ☐ GPS + photo capture working offline
- ☐ Sync queue with idempotency keys
- ☐ Version-based conflict detection on sync (never blind-overwrite)

- **Done when:** you can genuinely turn off Wi-Fi, complete a full field verification, turn Wi-Fi back on, and watch it sync without duplication — test this on a real device, not just localhost.

Phase 7 — Supporting Modules (can build in parallel once Phase 3 is stable)

- ☐ OCR document intake (Tesseract.js)
- ☐ Scheme Recommendation & Benefit Layering (rules-based, per official PS scope)
- ☐ Regional language switcher (English/Hindi/Telugu) on citizen status page
- ☐ Suggested Verification Order for officers (greedy grouping, honestly labeled)
- ☐ CSV/PDF export from dashboard

Phase 8 — Hardening & Testing

- ☐ Rate limiting on /auth/*, account lockout on repeated failed logins
- ☐ Unit tests for conflict engine and risk engine
- ☐ Integration tests for the sync endpoint (idempotency, version conflicts)
- ☐ Error monitoring wired up (Sentry or equivalent free tier)
- ☐ Security review against blueprint §17 checklist — only mark Trust Center items ACTIVE if true

Phase 9 — Demo Prep

- ☐ Synthetic dataset seeded (2 states, 6 districts, 2,000+ claims — blueprint §19)
- ☐ Hero claim + hero conflict seeded specifically for the demo
- ☐ Full offline demo rehearsed on actual presentation device/network
- ☐ 3-minute demo script rehearsed (blueprint §20)
- ☐ Backup demo recording captured in case of live failure

4. Conventions

- **Commits:** small, one logical change per commit, message describes *what* and *why*
- **Branches:** feature branches per module (feature/conflict-engine, feature/offline-sync), merge to main only when the phase’s “Done when” criteria are met
- **CHANGELOG.md:** update after every phase with what was built, what changed from the blueprint, and any known limitations
- **Known limitations file:** keep a running list (GPS spoofing risk, prototype-only thresholds, etc.) — these get surfaced honestly in the pitch, not hidden

5. What NOT to Build (this round)

Per blueprint §22 — do not spend time on: full voice assistant, complex satellite ML, blockchain, large deep-learning models, or any paid-tier service. Voice-assisted field notes specifically are cut because they require connectivity, which contradicts the offline-first hero demo.

6. Reference

Full technical detail (database schema, API routes, exact tech stack, security requirements) lives in `VanSetu_Blueprint_v2.md` — treat that as the source of truth for *what* to build; this file governs *how* and *in what order*.